

基于 Reduced Cost 的 Arc Fixing: 在 Arc-Time Branch-and-Price/Branch-Cut-and-Price 中的 思路与实现

2026 年 6 月 9 日

摘要

本文整理 arc-time-indexed branch-and-price/branch-cut-and-price 中常用的基于 reduced cost 的弧固定方法。该方法的核心思想是：对每一条 arc，先计算所有经过该 arc 的完整 source-to-sink path 的最小 reduced cost；若即使在最乐观补全下，任何使用该 arc 的整数解也不可能优于当前整数上界，则可安全地将该 arc 固定为零。该过程通常需要一次 forward shortest path 和一次 backward shortest path，即可对每条 arc 以常数时间计算其最优补全 reduced cost，因此非常适合 arc-time pricing 图中的大规模弧删减。

目录

1 背景：arc-time pricing 与 column 的含义	3
2 Reduced cost fixing 的基本直觉	3
3 为什么该规则是安全的？	4
4 如何快速计算 $\hat{c}_a$ ？	5
5 算法流程	5
5.1 输入与输出	5
5.2 伪代码	6
6 复杂度分析	6
7 与 column generation 的关系	7
8 不同模型结构下的改写	7
8.1 单机问题	7
8.2 Unrelated machines 或 machine-dependent pricing	7
8.3 路径数不固定、允许外包或拒单	8

9 数值实现中的注意事项	8
9.1 整数目标与容差	8
9.2 不可达节点与无穷距离	8
9.3 负 reduced-cost path 与下界解释	8
9.4 cut 和 branching 的影响	8
10 对算法设计的启发	9
11 小结	9

1 背景：arc-time pricing 与 column 的含义

在许多调度问题的 branch-and-price (B&P) 或 branch-cut-and-price (BPC) 算法中，单台机器上的一个 schedule 被表示为 pricing 图中的一条 source-to-sink path。以 identical parallel machines 为例，一个整数解可以看成选择 m 条机器路径：

$$P_1, P_2, \dots, P_m,$$

其中每条路径表示一台机器上的一个 schedule 或 pseudo-schedule。

在 arc-time-indexed 网络中，节点通常包含 job 和时间信息，例如

$$(i, t), \quad i \in \mathcal{J} \cup \{0\}, \quad t = 0, 1, \dots, T,$$

其中 0 是 dummy job。弧通常表示 “在某一时间从 job i 转移到 job j ”，例如

$$a = (i, j)^t.$$

若 j 在时间 t 完成，则这条弧会携带 job j 在完成时间 t 下的成本，或在 column generation 中携带相应的 reduced cost。

在 Dantzig-Wolfe reformulation 中，每一条 source-to-sink path $P \in \mathcal{P}$ 对应一个 column，记为 λ_P 。若 $q_a^P = 1$ 表示 path P 使用弧 a ，则 arc-flow 变量可以写成

$$x_a = \sum_{P \in \mathcal{P}} q_a^P \lambda_P.$$

因此，pricing 的任务就是在当前 RMP dual prices 下寻找 reduced cost 最小的 path：

$$\bar{c}^* = \min_{P \in \mathcal{P}} \sum_{a \in P} \bar{c}_a,$$

其中 $\bar{c}_a$ 是弧 a 的 reduced cost。

2 Reduced cost fixing 的基本直觉

假设当前已有一个整数可行解，其目标值为 UB。我们希望判断某条弧 a 是否可能出现在任何优于 UB 的整数解中。

核心问题是：

若强制某个整数解必须使用弧 a ，那么在最乐观补全下，这个整数解的目标值最小能到多少？

如果这个最乐观值仍然不能优于当前上界 UB，则弧 a 不可能出现在更优整数解中，可以安全删除。

设当前 RMP 的目标值为 Z_{RMP} 。在 Dantzig-Wolfe master 中， Z_{RMP} 可以理解为当前列池给出的 LP 下界。任意新加入的 path 对目标值的潜在改进由其 reduced cost 衡量。令

$$\bar{c}^* = \min_{P \in \mathcal{P}} \bar{c}(P)$$

表示全图中最短 path 的 reduced cost, 其中

$$\bar{c}(P) = \sum_{a \in P} \bar{c}_a.$$

对某条弧 a , 定义

$$\hat{c}_a = \min\{\bar{c}(P) : P \in \mathcal{P}, a \in P\},$$

即所有经过弧 a 的完整 source-to-sink paths 中的最小 reduced cost。

如果某个并机整数解使用了弧 a , 则至少有一条机器 path 必须经过该弧, 其 reduced cost 不小于 $\hat{c}_a$ 。其余 $m - 1$ 条机器 paths 即使都选择全图最短 path, 其 reduced cost 也不小于 $\bar{c}^*$ 。因此, 任何使用弧 a 的整数解, 其目标值均不小于

$$Z_{\text{RMP}} + \hat{c}_a + (m - 1)\bar{c}^*.$$

于是得到弧固定条件: 若

$$Z_{\text{RMP}} + \hat{c}_a + (m - 1)\bar{c}^* > \text{UB} - 1, \quad (1)$$

则弧 a 可以被安全固定为零:

$$x_a = 0.$$

这里的 $\text{UB} - 1$ 适用于目标函数取整数值的情形。如果目标函数不一定为整数, 则通常写为

$$Z_{\text{RMP}} + \hat{c}_a + (m - 1)\bar{c}^* \geq \text{UB} - \varepsilon,$$

其中 $\varepsilon > 0$ 是数值容差。

3 为什么该规则是安全的?

下面给出一个简洁证明。设 S 是任意使用弧 a 的整数解。由于是 m 台机器, S 可分解为 m 条 machine paths:

$$P_1, \dots, P_m.$$

不失一般性, 设 P_1 使用弧 a 。根据 $\hat{c}_a$ 的定义, 有

$$\bar{c}(P_1) \geq \hat{c}_a.$$

对其余 paths, 由 $\bar{c}^*$ 的定义, 有

$$\bar{c}(P_k) \geq \bar{c}^*, \quad k = 2, \dots, m.$$

因此, 使用弧 a 的整数解相对于当前 RMP 下界的 reduced-cost 补全贡献至少为

$$\hat{c}_a + (m - 1)\bar{c}^*.$$

于是该整数解的目标值满足

$$z(S) \geq Z_{\text{RMP}} + \hat{c}_a + (m - 1)\bar{c}^*.$$

如果式 (1) 成立, 则

$$z(S) > \text{UB} - 1.$$

若目标值为整数，则这意味着

$$z(S) \geq \text{UB},$$

即 S 不可能严格改进当前上界。因此，任何优于当前上界的整数解都不会使用弧 a ，可以安全令 $x_a = 0$ 。

4 如何快速计算 $\hat{c}_a$?

直接对每条弧 a 单独求

$$\min\{\bar{c}(P) : P \in \mathcal{P}, a \in P\}$$

会非常昂贵。arc-time 图通常是一个有向无环图 (DAG)，或者至少可以高效求 shortest path。因此可以先计算两类 shortest-path distances。

令 s 为 source 节点， q 为 sink 节点。对每个节点 $v \in \mathcal{V}$ ，定义：

$$d^+(v) = \text{从 } s \text{ 到 } v \text{ 的最短 reduced-cost 距离,}$$

$$d^-(v) = \text{从 } v \text{ 到 } q \text{ 的最短 reduced-cost 距离.}$$

其中 $d^+(\cdot)$ 通过一次 forward shortest path 得到， $d^-(\cdot)$ 通过一次 backward shortest path 得到。

对任意弧 $a = (u, v)$ ，有

$$\hat{c}_a = d^+(u) + \bar{c}_a + d^-(v). \quad (2)$$

这表示：若 path 必须经过弧 a ，那么最优 path 一定由三段组成：

$$\text{source 到 } u \text{ 的最短路} + \text{弧 } a + v \text{ 到 sink 的最短路.}$$

在 arc-time 调度图中，若弧记为 $a = (i, j)^t$ ，并且该弧从节点 $(i, t - p_i)$ 到节点 (j, t) ，则可写成

$$\hat{c}_{ij}^t = d^+(i, t - p_i) + \bar{c}_{ij}^t + d^-(j, t). \quad (3)$$

因此，在两组距离 d^+ 和 d^- 已经计算完成后，每条弧的 $\hat{c}_a$ 只需 $O(1)$ 时间即可得到。

5 算法流程

下面给出一个适用于 arc-time pricing 图的弧固定流程。该过程通常不会在每一次 column generation iteration 后都执行，而是每隔若干轮执行一次，例如每 K 轮执行一次，以平衡 fixing 收益与计算开销。

5.1 输入与输出

输入：

- 当前 pricing 图 $G = (\mathcal{V}, \mathcal{A})$;
- 当前 RMP 目标值 Z_{RMP} ;
- 当前整数上界 UB;

- 当前弧 reduced costs $\{\bar{c}_a : a \in \mathcal{A}\}$;
- 机器数 m ;
- 容差 ε 或整数目标修正项。

输出:

- 被固定为零的弧集合 $\mathcal{F} \subseteq \mathcal{A}$;
- 更新后的 pricing 图 $G' = (\mathcal{V}, \mathcal{A} \setminus \mathcal{F})$ 。

5.2 伪代码

Listing 1: 基于 reduced cost 的 arc fixing

```

Input: pricing graph  $G=(V,A)$ , arc reduced costs  $\text{cbar\_a}$ ,
      RMP value  $Z\_RMP$ , incumbent upper bound  $UB$ , number of machines  $m$ .
Output: fixed arc set  $F$ .

1. Compute forward shortest-path distances  $d\_plus(v)$  from source  $s$  to all  $v$ 
   in  $V$ .
2. Compute backward shortest-path distances  $d\_minus(v)$  from all  $v$  in  $V$  to
   sink  $q$ .
3. Let  $\text{cbar\_star} = d\_plus(q)$ , i.e., the shortest source-to-sink reduced cost
   .
4. Initialize  $F = \text{empty set}$ .
5. For each arc  $a=(u,v)$  in  $A$ :
    $\text{hat\_c\_a} = d\_plus(u) + \text{cbar\_a} + d\_minus(v)$ 
   if  $Z\_RMP + \text{hat\_c\_a} + (m-1)*\text{cbar\_star} > UB - 1$  then
     add  $a$  to  $F$ 
   end if
6. Remove all arcs in  $F$  from  $G$ .
7. Return  $F$  and the updated graph.

```

如果目标函数不保证为整数，应将第 5 步中的判断条件替换为

$$Z_{RMP} + \hat{c}_a + (m-1)\bar{c}^* \geq UB - \varepsilon.$$

6 复杂度分析

若 pricing 图为 DAG，则一次 forward shortest path 和一次 backward shortest path 均可在线性时间完成：

$$O(|\mathcal{V}| + |\mathcal{A}|).$$

随后，每条弧的 $\hat{c}_a$ 通过式 (2) 常数时间计算。因此整个 fixing 的复杂度为

$$O(|\mathcal{V}| + |\mathcal{A}|).$$

在标准 arc-time-indexed 调度网络中，弧数常为

$$|\mathcal{A}| = \Theta(n^2 T),$$

其中 n 是 job 数， T 是时间 horizon。因此 fixing 初始复杂度通常也是

$$\Theta(n^2 T).$$

不过，随着弧不断被固定，pricing 图会逐渐变小，后续 fixing 和 shortest path pricing 也会变快。

7 与 column generation 的关系

Reduced-cost fixing 通常嵌入 column generation 主循环中：

1. 解当前 RMP，得到 primal solution、dual prices 和 Z_{RMP} ；
2. 根据 dual prices 更新 pricing 图中的弧 reduced costs；
3. 解 pricing，若存在 negative reduced-cost path，则加入 column；
4. 每隔 K 轮，根据当前 Z_{RMP} 、UB 和弧 reduced costs 执行 arc fixing；
5. 若 pricing 无负 reduced-cost column，则当前节点 LP relaxation 求解完成。

该操作不会改变最优性，因为被删除的弧已经被证明不可能出现在任何优于当前上界的整数解中。需要注意的是，若后续得到更好的 UB，fixing 只会更强；若 Z_{RMP} 提升，也会更容易固定弧。

8 不同模型结构下的改写

8.1 单机问题

若只有一台机器，则一个整数解只对应一条 path。此时 $m = 1$ ，固定条件退化为

$$Z_{\text{RMP}} + \hat{c}_a > \text{UB} - 1.$$

或连续目标下

$$Z_{\text{RMP}} + \hat{c}_a \geq \text{UB} - \epsilon.$$

8.2 Unrelated machines 或 machine-dependent pricing

若每台机器 $k \in \mathcal{M}$ 有自己的 pricing 图 G_k 和 reduced costs，则对机器 k 上的弧 a ，应定义

$$\hat{c}_{k,a} = \min\{\bar{c}_k(P) : P \in \mathcal{P}_k, a \in P\},$$

$$\bar{c}_k^* = \min_{P \in \mathcal{P}_k} \bar{c}_k(P).$$

如果强制机器 k 使用弧 a ，其余机器 $\ell \neq k$ 取各自最短 reduced-cost path，则固定条件可写为

$$Z_{\text{RMP}} + \hat{c}_{k,a} + \sum_{\ell \in \mathcal{M} \setminus \{k\}} \bar{c}_\ell^* > \text{UB} - 1.$$

若该条件成立，则机器 k 的 pricing 图中弧 a 可以被固定为零。

8.3 路径数不固定、允许外包或拒单

如果 master problem 中不要求恰好选择 m 条 path，或者 job 可以通过外包、拒单等方式覆盖，则不能直接使用

$$(m-1)\bar{c}^*.$$

此时需要重新构造“剩余决策的最乐观 reduced-cost contribution”。一般形式为

$$Z_{\text{RMP}} + \hat{c}_a + \underline{R}_a > \text{UB} - \varepsilon,$$

其中 $\underline{R}_a$ 是在强制使用弧 a 后，其他机器路径、外包变量、拒单变量等剩余部分能够产生的最小 reduced-cost 补全下界。只要 $\underline{R}_a$ 是有效下界，该 fixing 规则仍然安全。

9 数值实现中的注意事项

9.1 整数目标与容差

若目标值为整数，使用 $\text{UB} - 1$ 可以避免数值误差下误删弧。若目标值为连续型，应使用容差：

$$Z_{\text{RMP}} + \hat{c}_a + (m-1)\bar{c}^* \geq \text{UB} - \varepsilon.$$

常用的 ε 可取 10^{-6} 到 10^{-5} ，具体取决于求解器精度和目标值尺度。

9.2 不可达节点与无穷距离

若 $d^+(u) = +\infty$ 或 $d^-(v) = +\infty$ ，说明弧 $a = (u, v)$ 不在任何完整 source-to-sink path 中，此时可直接删除该弧。

9.3 负 reduced-cost path 与下界解释

$\bar{c}^*$ 可能为负。此时 $(m-1)\bar{c}^*$ 表示其余机器 paths 在最乐观情况下可以继续降低目标值。因此固定条件仍然有效，但通常会变得更保守。只有当当前 Z_{RMP} 、 UB 和 $\hat{c}_a$ 足够强时，才能固定弧。

9.4 cut 和 branching 的影响

若 master 中加入了 cuts 或 branching constraints，只要这些约束的 dual contribution 可以分摊到弧 reduced cost 中，则 arc fixing 仍可直接使用。此时只需将弧 reduced cost 更新为包含所有 dual contribution 的值：

$$\bar{c}_a = c_a - \text{covering dual contribution} - \text{cut dual contribution} - \text{branching dual contribution}.$$

然后重新计算 d^+ 、 d^- 和 $\hat{c}_a$ 即可。

若 branching constraints 改变了 pricing 图结构，例如禁止某些 job transition，则应先删除对应弧，再在更新后的图上执行 shortest path 和 fixing。

10 对算法设计的启发

基于 reduced cost 的 arc fixing 特别适合以下情形：

- pricing 图很大，尤其是 $|\mathcal{A}| = \Theta(n^2T)$ 的 arc-time 网络；
- pricing 是 shortest path 或 DAG dynamic programming，forward/backward distances 容易计算；
- 当前已有较好的整数上界 UB；
- RMP 下界 Z_{RMP} 已经较强；
- 大量弧虽然在网络中存在，但不太可能出现在优质整数解中。

其主要作用不是直接改善 LP relaxation，而是减少 pricing 图规模，从而降低后续 pricing 和 branching 节点求解成本。对于时间离散化调度模型，这一点尤其重要，因为即使 job 数不大，时间 horizon T 较大时，网络弧数也可能非常庞大。

11 小结

Reduced-cost arc fixing 的关键公式是

$$Z_{\text{RMP}} + \hat{c}_a + (m - 1)\bar{c}^* > \text{UB} - 1.$$

其中 $\hat{c}_a$ 表示强制经过弧 a 的最短 source-to-sink path reduced cost， $\bar{c}^*$ 表示全图最短 path reduced cost。该条件的含义是：即使强制使用弧 a ，并且其余机器都采用最乐观 reduced-cost path，也无法得到优于当前上界的整数解。因此，弧 a 可以安全固定为零。

计算上，该方法只需一次 forward shortest path 和一次 backward shortest path，就能对每条弧以 $O(1)$ 时间计算 $\hat{c}_a$ 。因此，它是 arc-time B&P/BPC 中非常实用的网络缩减技术，尤其适合 shortest-path-based pricing。

参考文献

- [1] A. Pessoa, E. Uchoa, M. Poggi de Aragão, and R. Rodrigues. Algorithms over arc-time-indexed formulations for single and parallel machine scheduling problems. *Mathematical Programming Computation*, 2010.
- [2] D. Oliveira and A. Pessoa. An improved branch-cut-and-price algorithm for parallel machine scheduling problems. *INFORMS Journal on Computing*, 32(1):90–100, 2020.
- [3] D. Kowalczyk, R. Leus, C. Hojny, and S. Røpke. A flow-based formulation for parallel machine scheduling using decision diagrams. *INFORMS Journal on Computing*, 36(6):1696–1714, 2024.